

Search with AI

1. Overview

Goal:

Given a free-text keyword (e.g. “used Toyota Camry under 200k km in Al Ain”), automatically predict and apply the correct structured filters on the search page.

This presumes we only target the **X most important filters** in a certain category not the entirety of dubizzle filters. If a keyword is outside those important filters the **remaining filters should be applied as a keyword**.

Motivation:

- Improve relevance and conversion by bridging natural language and structured filters.
- Reduce friction and dependency on manual filter selection.
- Establish a reusable text-to-filter prediction framework for all categories.

Open Questions:

- What is an acceptable response time in case of new keyword combination how long is it allowed to take max to retrieve the filters?
 - Answer: Ideally it should be below 1 second, for image based searches it can be higher
- Is it acceptable to only target important filters?
 - Answer: Initially possible, goal is to cover all

2. Architecture / Components

Phase	Description	Outcome
Indexing	Index listings into Typesense in a generic flat format	Index and embed listing data to insert it into typesense for hybrid search lookup
Search Service	Setup a new service using FastAPI that will manage the search to filter requests	Allow for async request structure flow to process parallel requests

Query Cache	Store all keywords to filter combinations	Allow quick lookup for exact or similar matches in future to filters to either return direct filters or smaller prompt
Filter mapping	Transform prompt data into client search_state	Using HBS map the returned prompt response filters from Typesense filter names into SearchState
Monitoring / Evaluation	Setup monitoring and evaluation framework	Have an async job review prompts/images and evaluate the filters are correct
Tracking	Setup tracking events from search to DPV	Have reliable tracking on CVR/CTR including the model, provider etc.

The main problem statement is that we can do a LLM call with all our filters and categories and ask it to predict the filters but that leads to two problems:

- Prompt Context size is huge
- Response time is slow
- Accuracy is bad because overloaded prompt

We can front load the category prediction to reduce the filter size but this has the following issues:

- One LLM call to predict the right category is one extra hop before getting the filters
- The category tree is huge and costly to send

To improve up on both issues the best would be to send the filters and categories together but limited to the user query or image. This can be done in two ways:

- Embed user query or image

- Do a listing lookup using those vectors and retrieve the remaining facet count, which will give us top 5 facets for whatever is related to what the user has asked

This improves the following things:

- Reduce filter scope
- Reduce AI call to determine category to just one embedding call plus cheap search call
- More reduced context size for filter prediction as facet count is sorted by actual listings remaining
- More accurate results because filters send are just selected ones

1: Indexing

We will use Typesense to determine the facet values matching the user query.

For this we will use Typesense [Hybrid Search](#) as it supports:

- Vector Search using Query Embedding and Listing Embedding
- Keyword Search using normal Text Match

Given this we have lexical matches and semantic matches covered, we can get hopefully reliable facet counts.

To index the listing we will follow a flat format, without any nested keys like we do on Algolia.

For example:

- details.Price.value.en
- category_v2.slug_paths

Instead:

- price
- category

We will apply this change for all the current Algolia keys. For all tags we will index them by tag group name and the value will be tag value.

To do the vector search we will introduce 3 initial vectors, which we can extend or version in future:

- buyer_vector - Transformed listing attributes with few keywords reflecting buyer intent
- seller_vector - Transformed listing attributes with few keywords reflecting seller intent
- image_vector - Pure image embedding

In future we can combine text and image into another vector. The idea of those vectors is to cover searches saying things like “I want a car”, “Car for sale”, or “car”.

For the embeddings we will initially rely on this model: [voyage-multimodal-3.5](#)

Open Questions:

- As we are using a flat structure, how do we handle new fields coming in, those will fail to insert the whole listing on Typesense? Qdrant may be better fit

2: Search Service

The Search Service is the actual service executing the vector/semantic search to retrieve filter values from Typesense.

Few considerations:

- We need to be able to execute parallel requests to reduce response time
- We need to install a few additional dependencies
- We need to allow for long running requests, latency will be high

We decided to:

- Create a new FastAPI service to have async parallel requests
- Long running tasks
- Separate repo with its own dependencies as they are different from the usual workload of HBS

We will have one API only for the moment in this service which is:

POST <https://dubizzle.com/svc/search/api/v1/filters/>

The API will take the following payload / this will be multipart request not JSON just to show:

```
1 {  
2   "query": "bmw",  
3   "variant": "original_950" (Optional),  
4   "image": (Multipart request) (Optional),  
5   "audio": (Multipart request) (Optional)  
6 }
```

The response will be:

```
1 {  
2   "query": "bmw",  
3   "filters":  
4     {  
5       "keyword": "bmw"
```

```

6   }
7 }

```

The most important point is that filters will follow search_state standard of HBS.

This endpoint is similar to:

https://dubai.dubizzle.com/classified/svc/buyer/api/v1/suggestions/filters/?query=bmw&variant=original_950

Just that we have made it POST, are accepting media formats and also switch the service to have async processing support.

The request flow is detailed in the flow diagram below:



Link to Diagram: <https://mermaid.ai/d/862c3466-66dc-4004-a2d0-5d6a811f043e>

Mermaid Schema:

```

1 flowchart TD
2   Start([User Submits Search]) --> HasImage{Has Image?}
3
4   %% Image Flow
5   HasImage -->|Yes| ParallelImage[/Parallel Calls/]
6   ParallelImage --> ImageEmbed[Voyage: Get Image Embedding]
7   ParallelImage --> LLMQuery[LLM: Get Query from Image]
8   ParallelImage --> CompressImage[LLM: Compress to Search Term]
9
10  ImageEmbed --> SetImageType[vectorType = 'image']
11  LLMQuery --> SetSearchQuery1[searchQuery = geminiQuery or
userQuery]
12  CompressImage --> CompressedTerm1[compressedTerm]
13
14  SetImageType --> MergeImage((Join))
15  SetSearchQuery1 --> MergeImage
16  CompressedTerm1 --> MergeImage

```

```

17
18     %% Text Flow
19     HasImage -->|No| ParallelText[/"Parallel Calls"/]
20     ParallelText --> TextEmbed[Voyage: Get Text Embedding]
21     ParallelText --> CompressText[LLM: Compress to Search Term]
22
23     TextEmbed --> SetTextType[vectorType = 'text']
24     CompressText --> CompressedTerm2[compressedTerm]
25
26     SetTextType --> MergeText((Join))
27     CompressedTerm2 --> MergeText
28
29     %% Start compressed embedding in background
30     MergeImage --> StartCompressedEmbed{compressedTerm<br/>differs
from query?}
31     MergeText --> StartCompressedEmbed
32
33     StartCompressedEmbed -->|Yes| BackgroundEmbed[/Voyage: Start
Text Embedding<br/>for compressed term<br/>- runs in background/]
34     StartCompressedEmbed -->|No| SkipCompressed[Skip compressed
embedding]
35
36     BackgroundEmbed --> CheckVectorType
37     SkipCompressed --> CheckVectorType
38
39     %% Contextual Facets - Different paths for text vs image
40     CheckVectorType{vectorType?}
41
42     CheckVectorType -->|text| ParallelVectorSearch[/"Typesense:
Parallel Vector Search<br/>text_vector + buyer_vector<br/>(single
multiSearch call)"/]
43     CheckVectorType -->|image| ImageVectorSearch[Typesense: Vector
Search<br/>image_vector only]
44
45     ParallelVectorSearch --> MergeFacets[Merge unique
facets<br/>from both results]
46     MergeFacets --> HasFacets1
47     ImageVectorSearch --> HasFacets1
48
49     HasFacets1{Has facet<br/>values?}
50
51     %% Fallback: Compressed Embedding
52     HasFacets1 -->|No| HasCompressedPromise{Has
compressed<br/>embedding promise?}
53     HasCompressedPromise -->|Yes| AwaitCompressed[Await compressed
embedding]
54     AwaitCompressed --> CompressedParallelSearch[/"Typesense:
Parallel Vector Search<br/>text_vector + buyer_vector<br/>with
compressed term"/]
55     CompressedParallelSearch --> HasFacets2{Has facet<br/>values?}
56
57     HasCompressedPromise -->|No| SkipToLLM
58     HasFacets2 -->|No| SkipToLLM[No facets available]
59
60     %% LLM Filter Generation
61     HasFacets1 -->|Yes| LLMFilters
62     HasFacets2 -->|Yes| LLMFilters
63
64     LLMFilters{Has image?}
65     LLMFilters -->|Yes| LLMWithImage[LLM: Generate Filters<br/>with
image + facets + query]
66     LLMFilters -->|No| LLMTextOnly[LLM: Generate Filters<br/>with
facets + query]
67
68     LLMWithImage --> ShowResults
69     LLMTextOnly --> ShowResults

```

```

70      SkipToLLM --> ShowResultsNoFilter[Show Search Results<br/>query
    only, no AI filters]
71
72      %% User-facing actions
73      ShowResults[Show Search Results<br/>with AI-suggested filters]
74
75      ShowResults --> ShowSuggestion[Display AI Suggested Query<br/>in
    purple banner]
76      ShowResultsNoFilter --> End
77
78      ShowSuggestion --> End([User can apply suggestion<br/>or
    continue browsing])
79
80      %% Styling
81      classDef voyage fill:#e1f5fe,stroke:#01579b
82      classDef llm fill:#fce4ec,stroke:#880e4f
83      classDef typesense fill:#e8f5e9,stroke:#1b5e20
84      classDef decision fill:#fff3e0,stroke:#e65100
85      classDef userAction fill:#e8eaf6,stroke:#283593
86      classDef parallel fill:#fff8e1,stroke:#f57f17
87
88      class ImageEmbed,TextEmbed,BackgroundEmbed,AwaitCompressed
    voyage
89      class
    LLMQuery,CompressImage,CompressText,LLMWithImage,LLMTextOnly llm
90      class
    ParallelVectorSearch,ImageVectorSearch,CompressedParallelSearch
    typesense
91      class
    HasImage,HasFacets1,HasFacets2,StartCompressedEmbed,HasCompressedPromi
    se,CheckVectorType,LLMFilters decision
92      class ShowResults,ShowSuggestion,ShowResultsNoFilter,End
    userAction
93      class ParallelImage,ParallelText,MergeFacets parallel

```

The high-level logic is this:

1. Step - Prepare first Typesense request all in parallel:

- If image search we will use pillow to reduce size of the image to save cost and improve LLM response time
- Embed but separately the query text and image
- Request LLM to compress the inputs which includes text and image (not embedding but original request) into a keyword search friendly query

Note: We will only wait for the first item above to be finished to continue to step 2, the LLM request can still be pending while we transition to the next step.

2. Step - Try to get facet counts

- Use the original text query and the embeddings to do a listing search but instead of retrieving listing get the facet counts
 - If only text based query was done:
 - Do two requests on Typesense using multisearch:
 - One against buyer_vector

- One against seller_vector
 - Both should include the original keyword inside the “q” parameter for a pure text based match
 - If image based query was done:
 - Do one request on Typesense:
 - Use the image embedding against image_vector
 - Use the original text as “q” parameter for a text based match
 - Typesense will do a Hybrid Search as mentioned before, it will not discard results where the keyword is not found if the image is a match.
 - Post process the facets to remove all empty facets etc., if the result is that not facets were found go to the next step otherwise skip and go to the last step with the filters
3. Step - Try to get facet count using compressed query - **Only if previous step did not yield any results**
- There is one important parallel call which we started in step 1 which is to “compress” the user inputs into a more understandable keyword search, this request is going on in parallel the whole time. If ever it finishes before coming to step 3, we should not wait to do the following:
 - Here we await the LLM call to compress the user inputs, and if available **embed** that compressed search term again.
 - We will then repeat what was done in step 2 using those new inputs:
 - Do two requests on Typesense using multisearch with the **embedded compressed search term**:
 - One against buyer_vector
 - One against seller_vector
 - Both should include the **compressed** keyword inside the “q” parameter for a pure text based match
4. Step - Get filters
- Assuming that either step 2 or 3 have yielded any results, we will construct a prompt and pass that to an LLM to pick the “best matches” and whatever does not fit leave inside the keywords as remainder. We will pass on the image if it was an image search.
 - If no facet results came back the whole query will go back to keyword search.
5. Step - Map filters to search state
- a. As the LLM will construct the filters matching to the facets we have defined on Typesense, we will need to convert them back to an actual FE understandable search_state

- b. For this we will do one request to HBS which will map the attributes name_en to filter name (not search_name), FE will handle actual conversion from search state to Algolia Params or for community pass this search state into the listings API of HBS.

Caveats with this solution:

- As the filters are provided based on the facet we could find on a similarity match against our own inventory, it will not be able to detect filters if we don't have listings matching the query in the first place. This is different to how a pure filter schema approach works where we try to predict the category from the query and pass all filters to the LLM.

Open points:

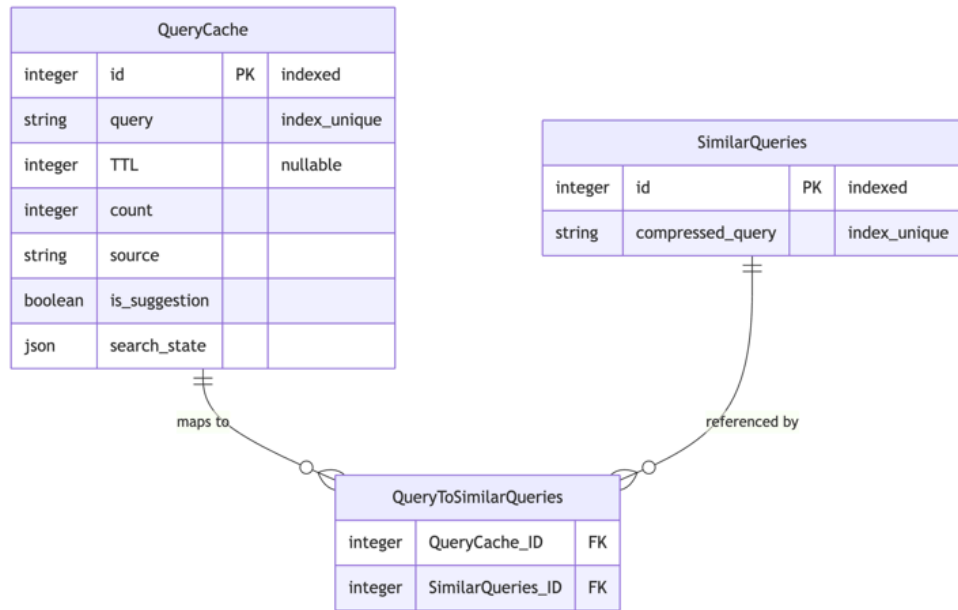
- We are reducing image size with pillow, can we keep it in memory till the request finishes as we need to send it to different LLM requests or should use redis to store?
- If the image search yields no filters, is it ok to apply a keyword or should we just “suggest it”?
- Image search will not allow “looking similar feature” on the actual LPV till we transition to Typesense or find a solution to rank those results higher by a Typesense lookup.
- Voice based solution is open but finally we will translate the voice to text and just go through the normal text pipeline. Not priority from product side.

3: Query Cache

As the process of using LLM for every search query is expensive we want to build up a cache so that future queries are cheaper in two ways:

- Exact matches should result in same filters can be stored as key value, where key is the keywords and value the filters
- Similar searches can be used to more easily predict the filters

To handle this we will create a separate database and tables as the table size will be massive. Here the structure:



Tables:

QueryCache:

- id ⇒ Primary Key
- query ⇒ The query the user searched for
- TTL ⇒ Null by default, will be set to some value if filters get changed
- count ⇒ Number of times this query was used
- source ⇒ Will be enum initially “AI” and “BI” will be there
- is_suggestion ⇒ Based on our current HBS logic we will mark this either true or false; not used initially but will be used to serve suggestions later on
- search_state ⇒ The filters that need to be applied

SimilarQueries:

- id ⇒ Primary Key
- string ⇒ Compressed Query we requested from LLM
- This is a separate table because similar input texts can lead into the same “compressed query” like “lamp under 300” and “lamp for 300” can both end up with “lamp price 300”

QueryToSimilarQueries:

- Many to many table to link the above two tables

Open questions:

- Should we have a separate table FilterSet, this will be just referenced by the QueryCache, for the same reason of SimilarQueries

- Invalidation of search_state when filter changes will be expensive
- If we need to reembed, we need to scan the full table same as we need to do on listings

Mermaid Schema:

```

1  erDiagram
2      QueryCache {
3          integer id PK "indexed"
4          string query "index_unique"
5          integer TTL "nullable"
6          integer count
7          string source
8          boolean is_suggestion
9          json search_state
10     }
11
12     QueryToSimilarQueries {
13         integer QueryCache_ID FK
14         integer SimilarQueries_ID FK
15     }
16
17     SimilarQueries {
18         integer id PK "indexed"
19         string compressed_query "index_unique"
20     }
21
22     %% Relationships
23     QueryCache ||--o{ QueryToSimilarQueries : "maps to"
24     SimilarQueries ||--o{ QueryToSimilarQueries : "referenced by"

```

Process of how this table is going to be used initially:

- Every request to our new endpoint on search service will hit the QueryCache and just check for an exact match
- If yes it will directly return the search_state
- If no the above mentioned process of how to determine filters will be executed

Storing Process:

In the Search Service part we did not explain how this data will be stored.

- We will store this data by spawning a separate async task which will receive all inputs excluding the image and the embeddings
- We will simply store the new query into the QueryCache table
- For the SimilarQueries table we will do a lookup
 - If the compressed query already exists we will just add an entry into the many to many table
 - If the compressed query does not exist we will create a new record and then add a link in the many to many table

After this we will have another task which is to store this data inside two collections on Typesense:

- All the QueryCache items will be stored in a suggestions index including the embedding
- All the SimilarQuery items will be stored in a similar_query index including the embedding

In V1 the above two collections wont be used, but in future we can use them to find similar queries. For example:

User searches for “lamp under 500”, instead of doing an LLM call to predict filters given all the listings and their facet counts matching this search we will just check if we have a similar query “lamp for 300” for which we already store { price: 300 } as filters. We can then let AI predict the filters from that.

4: Filter Mapping

5: Monitoring & Evaluation

6: Tracking

Extensions (not V1)

1. Similar keyword lookup to speed up predictions

Objective:

Embed user queries and compressed queries

POC Approach:

Lookup based on embedding vector similar searches and use them as example to LLM call instead of using unrelated examples with facets

2. Dynamic Schema Adaptation

Objective:

Auto-detect and adapt to new filters added in backend categories.

POC Approach:

We will set a TTL on query cache to expire certain keywords

3. Feedback & Self-Learning

Objective:

Continuously improve prediction quality based on user corrections and click data.

POC Approach:

Collect feedback and clicks to identify wrong predictions.

4. Dynamic Filters

Objective:

Instead of having hardcoded filters and options according what category we are on, we should just get the available filters remaining given my current keyword / filter selection and display this.

POC Approach:

On global search change the static filters which are minimal to be dynamic based on the current Facets remaining after keyword search and show them dynamically.